Coding: ASP.Net Core Standards

ASPDotNetCore

```
---
applyTo: "**"
---

# ASP.NET Core 9.0 Essential Standards

## Table of Contents
- [Introduction](#introduction)
- [Code Organization](#code-organization)
- [Coding Conventions](#coding-conventions)
- [ASP.NET Core Best Practices](#aspnet-core-best-practices)
  - [Controller Design](#controller-design)
  - [Middleware Usage](#middleware-usage)
  - [Dependency Injection](#dependency-injection)
- [Performance Optimization](#performance-optimization)
- [Security Standards](#security-standards)
- [Testing Guidelines](#testing-guidelines)
- [References](#references)

## Introduction

This document outlines essential standards and best practices for ASP.NET Core 9.0 development. Following these standards will help ensure code quality, maintainability, and performance.

## Code Organization

- **Follow Clean Architecture principles**
  - Separate into layers (Presentation, Application, Domain, Infrastructure)
  - Use feature folders over type-based organization

- **Keep controllers thin** - Move business logic to services
- **One class per file** - Each file should contain only one class
- **Limit file size** - Keep files under 500 lines when possible

## Coding Conventions

### Naming Conventions

- **PascalCase** for:
  - Classes, Methods, Properties, Public fields, Namespaces, Enum values
- **camelCase** for:
  - Local variables, Method parameters, Private fields (with underscore prefix)
- **Prefix interfaces with 'I'**: `IUserService`

```csharp
public class UserService : IUserService
{
    private readonly IUserRepository _userRepository;

    public async Task<User> GetUserByIdAsync(int userId)
    {
        return await _userRepository.GetByIdAsync(userId);
    }
}
```

Asynchronous Programming

- **Use async/await consistently** - Don't mix sync and async code
- **Add Async suffix** to asynchronous methods
- **Return Task or Task<T>** - Never use void for async methods except event handlers
- **Use ConfigureAwait(false)** in libraries

```csharp
// Good
public async Task<User> GetUserByIdAsync(int userId)
{
    return await _userRepository.GetByIdAsync(userId);
}

// Bad
public User GetUserById(int userId)
{
    return _userRepository.GetByIdAsync(userId).Result; // Blocks thread
```
```

```
}
```

ASP.NET Core Best Practices

Controller Design

- **Return appropriate ActionResult types** - Use IActionResult or ActionResult<T>
- **Use attribute routing** over conventional routing
- **Follow RESTful conventions** - Use appropriate HTTP methods
- **Validate model state** before processing requests

```csharp
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    private readonly IUserService _userService;

    [HttpGet("{id}")]
    public async Task<ActionResult<UserDto>> GetUser(int id)
    {
        var user = await _userService.GetUserByIdAsync(id);
        if (user == null) return NotFound();
        return _mapper.Map<UserDto>(user);
    }

    [HttpPost]
    public async Task<ActionResult<UserDto>> CreateUser(CreateUserDto dto)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);
        var user = await _userService.CreateUserAsync(dto);
        return CreatedAtAction(nameof(GetUser), new { id = user.Id },
            _mapper.Map<UserDto>(user));
    }
}
```

Middleware Usage

- **Order middleware correctly** - Consider execution order
- **Keep middleware focused** - Single responsibility
- **Use app.UseWhen for conditional middleware**
- **Use Global exception handling middleware**
- **Include OpenAPI (middleware) for API documentation using Swagger**

```csharp
// In Program.cs
app.UseExceptionHandler("/error");
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.UseResponseCompression();
app.MapControllers();
```

For HSTS configuration

```csharp
httpContext.Response.Headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains; preload";
```

Dependency Injection

- **Register services with appropriate lifetimes**:
  - Transient: Created each time requested
  - Scoped: Created once per request
  - Singleton: Created once for application lifetime
- **Use constructor injection** over property injection
- **Register interfaces, not implementations**

```csharp
// Service registration
public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddApplicationServices(
        this IServiceCollection services)
    {
        services.AddScoped<IUserService, UserService>();
        services.AddSingleton<ICacheService, CacheService>();
        services.AddTransient<IEmailSender, EmailSender>();
```

```
        return services;
    }
}
```

Performance Optimization

- **Cache aggressively**
  - Use Memory Cache for frequently accessed data
  - Implement Distributed Cache for web farms
  - Set appropriate cache durations

```csharp
// Memory caching example
public async Task<List<Category>> GetCategoriesAsync()
{
    var cacheKey = "AllCategories";
    if (!_memoryCache.TryGetValue(cacheKey, out List<Category> categories))
    {
        categories = await _categoryRepository.GetAllAsync();
        _memoryCache.Set(cacheKey, categories, TimeSpan.FromMinutes(10));
    }
    return categories;
}
```

- **Make hot code paths asynchronous**
  - Use async/await for I/O operations
  - Avoid blocking calls that lead to thread pool starvation

- **Optimize data access**
  - Use no-tracking queries for read-only operations
  - Retrieve only necessary data
  - Implement pagination for large collections

```csharp
// Efficient EF Core query
public async Task<List<ProductSummaryDto>> GetProductSummariesAsync(int page, int pageSize)
{
    return await _dbContext.Products
        .AsNoTracking()
        .Where(p => p.IsActive)
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .Select(p => new ProductSummaryDto
        {
            Id = p.Id,
            Name = p.Name,
            Price = p.Price
        })
        .ToListAsync();
}
```

- **Minimize large object allocations**
  - Use ArrayPool<T> for large arrays
  - Implement streaming for large files

- **Implement response compression**
  - Compress HTTP responses to reduce payload size

Security Standards

- **Use HTTPS** - Require HTTPS in production

```csharp
builder.Services.AddHsts(options =>
{
    options.Preload = true;
    options.IncludeSubDomains = true;
    options.MaxAge = TimeSpan.FromDays(365);
});
app.UseHsts();
```


- **Implement proper authentication**
  - Use ASP.NET Core Identity or JWT for APIs
  - Apply [Authorize] attributes appropriately

```csharp
// Policy-based authorization
```

```csharp
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Administrator"));
});

[Authorize(Policy = "AdminOnly")]
public async Task<IActionResult> AdminAction()
{
    // Implementation
}
```

- **Set fallback authorization policy**
  - Require authentication by default for all endpoints
  - Use AllowAnonymous for public pages

```csharp
// Require authentication for all endpoints by default
builder.Services.AddAuthorization(options =>
{
    options.FallbackPolicy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build();
});

// Mark specific pages as public
[AllowAnonymous]
public class IndexModel : PageModel
{
    // Implementation
}
```

- **Implement role-based authorization**
  - Define clear roles (e.g., Users, Managers, Administrators)
  - Add roles to Identity services

```csharp
// Add role services to Identity
builder.Services.AddDefaultIdentity<IdentityUser>()
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();

// Create and assign roles
var roleManager = serviceProvider.GetService<RoleManager<IdentityRole>>();
await roleManager.CreateAsync(new IdentityRole("Administrators"));
await userManager.AddToRoleAsync(user, "Administrators");
```

- **Implement resource-based authorization**
  - Create authorization handlers for resource operations
  - Register handlers with dependency injection
  - Protect user data by tying it to user IDs

```csharp
// Authorization handler example
public class ResourceOwnerAuthorizationHandler
    : AuthorizationHandler<OperationAuthorizationRequirement, Resource>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        OperationAuthorizationRequirement requirement,
        Resource resource)
    {
        if (context.User.GetUserId() == resource.OwnerId)
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}

// Register handlers
services.AddScoped<IAuthorizationHandler, ResourceOwnerAuthorizationHandler>();
services.AddSingleton<IAuthorizationHandler, AdminAuthorizationHandler>();

// Check authorization in controllers/pages
var isAuthorized = await _authorizationService.AuthorizeAsync(
    User, resource, Operations.Update);
if (!isAuthorized.Succeeded)
{
```

```
    return Forbid();
}
```

- **Validate all input**
  - Use data annotations or fluent validation
  - Sanitize user input to prevent XSS

```csharp
public class CreateUserDto
{
    [Required]
    [StringLength(100, MinimumLength = 2)]
    public string Name { get; set; }

    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [StringLength(100, MinimumLength = 8)]
    public string Password { get; set; }
}
```

- **Implement CSRF protection**
  - Use anti-forgery tokens for forms
  - Apply [ValidateAntiForgeryToken] to POST actions

- **Set security headers**
  - Add Content-Security-Policy, X-XSS-Protection, etc.

```csharp
app.Use(async (context, next) =>
{
    context.Response.Headers.Add("X-Content-Type-Options", "nosniff");
    context.Response.Headers.Add("X-Frame-Options", "DENY");
    context.Response.Headers.Add("X-XSS-Protection", "1; mode=block");
    context.Response.Headers.Add("Referrer-Policy", "strict-origin-when-cross-origin");
    context.Response.Headers.Add("Content-Security-Policy",
        "default-src 'self'; script-src 'self' https://trusted-cdn.com");

    await next();
});
```

Application Health

- Use health check endpoint to monitor status of application dependencies
- Provide custom response to see details about each dependency

```csharp
app.MapHealthChecks(
    "/api/health",
    new HealthCheckOptions
    {
        Predicate = _ => true,
        ResponseWriter = async (context, report) =>
        {
            context.Response.ContentType = "application/json";
            var response = new
            {
                status = report.Status.ToString(),
                checks = report.Entries.Select(x => new
                {
                    name = x.Key,
                    status = x.Value.Status.ToString(),
                    exception = x.Value.Exception?.Message,
                    duration = x.Value.Duration.ToString()
                })
            };
            await context.Response.WriteAsync(JsonSerializer.Serialize(response));
        }
});
```

Testing Guidelines

- **Follow AAA pattern** - Arrange, Act, Assert
- **Use mocking frameworks** - Moq or NSubstitute for isolating dependencies
- **Test one concern per test** - Keep tests focused

```csharp
```

```
// Unit test example
[Fact]
public async Task GetUserById_WithExistingId_ReturnsUser()
{
    // Arrange
    var userId = 1;
    var expectedUser = new User { Id = userId, Name = "Test User" };
    var repositoryMock = new Mock<IUserRepository>();
    repositoryMock.Setup(repo => repo.GetByIdAsync(userId))
        .ReturnsAsync(expectedUser);
    var userService = new UserService(repositoryMock.Object);

    // Act
    var result = await userService.GetUserByIdAsync(userId);

    // Assert
    Assert.NotNull(result);
    Assert.Equal(userId, result.Id);
}
```

- **Use integration tests** for testing the full request pipeline
- **Isolate test data** - Each test should have its own data

References

- [ASP.NET Core Best Practices](https://learn.microsoft.com/en-us/aspnet/core/fundamentals/best-practices?view=aspnetcore-9.0)
- [C# Coding Conventions](https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions)
- [ASP.NET Core Performance Best Practices](https://learn.microsoft.com/en-us/aspnet/core/performance/performance-best-practices?view=aspnetcore-9.0)
- [ASP.NET Core Security Best Practices](https://learn.microsoft.com/en-us/aspnet/core/security/authorization/secure-data?view=aspnetcore-9.0)
-